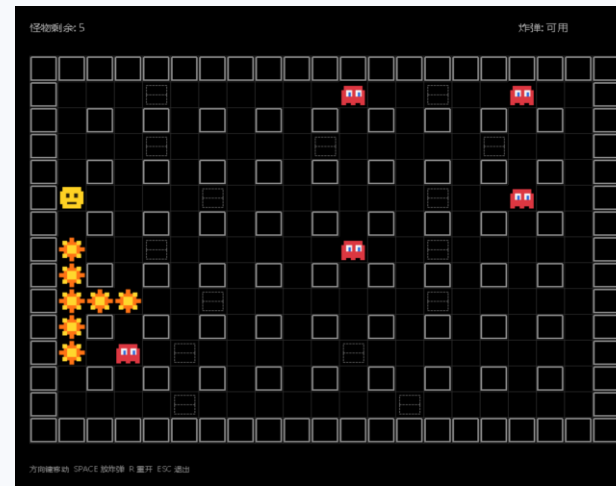
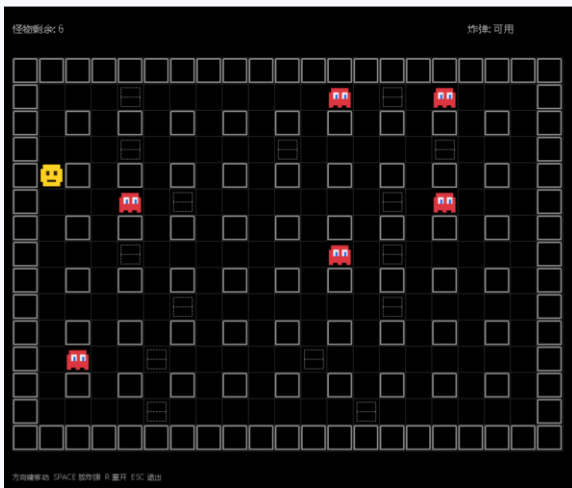


简易炸弹人

EasyBomber

基于 C++17 与 EasyX 的课程项目



游戏主体为简单的炸弹人玩法，包含玩家（Player）、怪物（Monster）、炸弹（Bomb）、地图（Map）等元素。

在游戏中，玩家需要放置炸弹炸毁砖块、消灭全部 6 只怪物来获得胜利，同时也要躲避怪物的触碰和自身爆炸的波及。怪物沿固定路线往返巡逻，遇到未炸毁的砖块会原地等待。

炸弹在放置 2 秒后引爆，爆炸向上下左右各传播 2 格，实线墙会阻挡爆炸，虚线砖块会被炸毁清除。

游戏同时只能放置一颗炸弹，玩家不小心被自己的爆炸炸到也会失败。



EasyX Graphics Library 是针对 Visual C++ 的免费绘图库，支持 VC6.0 ~ VS2022，简单易用，学习成本极低，可以帮助使用 C/C++ 语言的程序员快速上手图形和游戏编程。EasyX 提供了绘图、文字处理、消息处理、图片处理等诸多功能的函数，使用时需引用 `graphics.h` 或 `easyx.h` 头文件。具体用法参见官方文档：
<https://docs.easyx.cn/zh-cn/intro>



EasyX 核心能力

绘图函数 — putimage() / 几何图形 / 颜色填充 / 像素操作

文字处理 — outtextxy() / 字体样式设置 / 透明背景模式

消息处理 — ExMessage / 键盘事件 / 窗口事件 / 鼠标事件

图片加载 — loadimage() / 支持 PNG / JPG / BMP 格式

双缓冲 — BeginBatchDraw() / FlushBatchDraw() / EndBatchDraw()

EasyBomber 中的使用

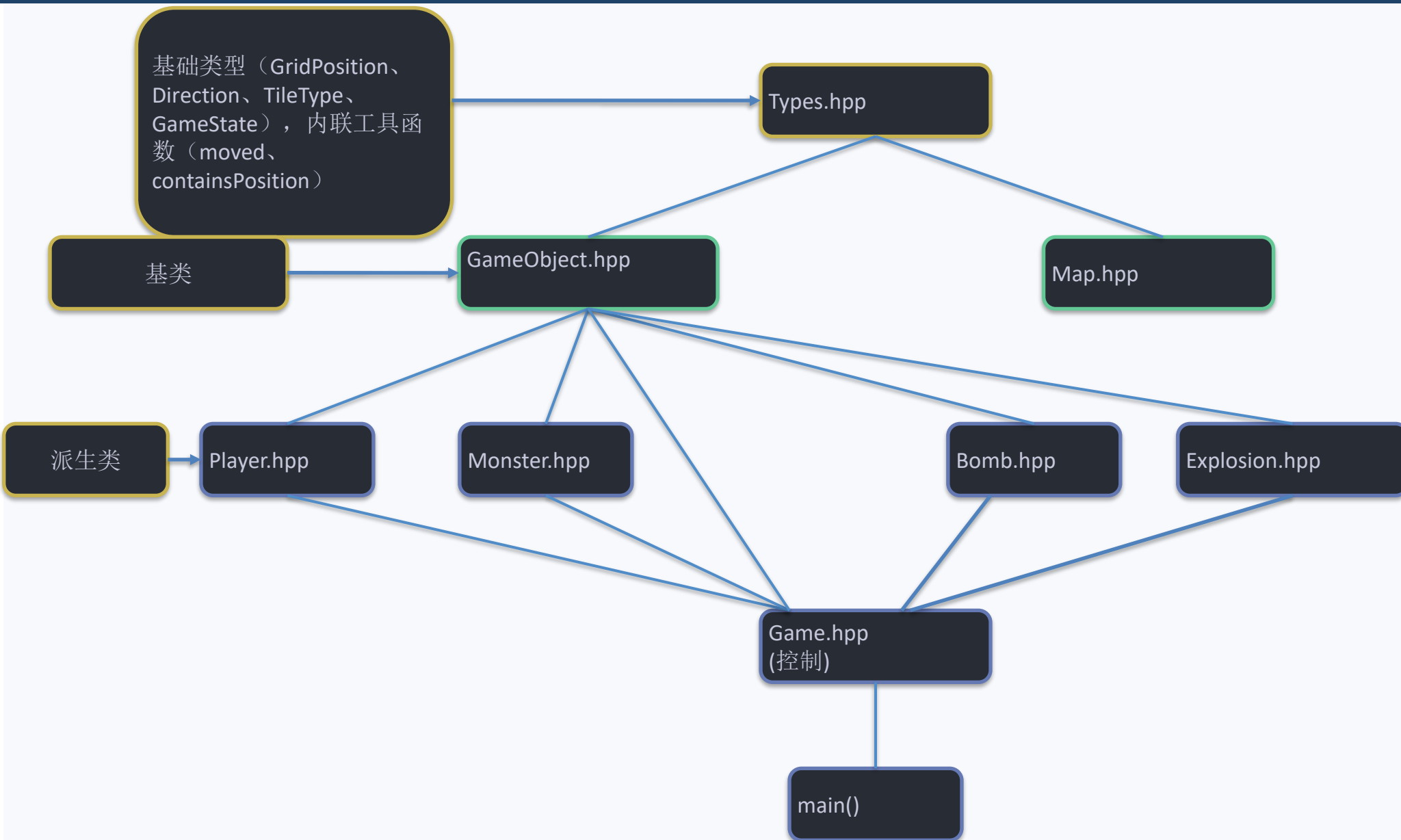
EasyBomber 入口 — Game::run()

```
#include <graphics.h> // EasyX 头文件
int Game::run() {
    initgraph(WIN_WIDTH, WIN_HEIGHT);
    SetWindowText(GetHWnd(), L"简易炸弹人");
    setbkcolor(BLACK);
    BeginBatchDraw(); // 开启双缓冲

    while (running) {
        input(); update(dt); draw();
        FlushBatchDraw(); // 双缓冲呈现
        Sleep(8);
    }

    EndBatchDraw(); // 关闭双缓冲
    closegraph(); return 0;
}
```

2 —整体架构 — 文件依赖与继承关系





```
// 抽象基类, 所有动态对象的公共基础
class GameObject {
protected:
    GridPosition pos;        // 格子坐标
    bool active = true;      // 生存标记
public:
    GameObject(GridPosition p) : pos(p) {}
    virtual ~GameObject() = default; //虚析构函数

    virtual void update(int dt, const Map& map) = 0;
    virtual void draw() const = 0;
    GridPosition getPos() const { return pos; }
    bool isActive() const { return active; }
    void remove() { active = false; }
};
```

- 定义了两个纯虚函数 `update(int dt, const Map& map)` 和 `draw()`, 强制派生类实现
- `GridPosition pos` 记录格子坐标, `bool active` 标记存活状态
- `remove()` 只把 `active` 设为 `false`, 不真的 `delete`。由 `Game` 统一在 `clearObjects()` 中清理
- 虚析构函数保证通过基类指针 `delete` 派生类时正确释放



```
// 键盘驱动移动, 无需每帧更新
class Player : public GameObject {
public:
    Player(GridPosition p);
    void update(int, const Map&) override {}
    void draw() const override;
    bool tryMove(Direction dir, const Map& map,
                  const vector<GridPosition>& bombs);
};

bool Player::tryMove(Direction dir, const Map& map,
                     const vector<GridPosition>& bombs)
{
    auto next = moved(pos, dir);
    if (!map.canWalk(next) ||
        containsPosition(bombs, next)) return false;
    pos = next; return true;
}
```

- 空的update() 确保由键盘事件直接调用, 使得每一帧自己更新
- tryMove(dir, map, bombs): 先算目标格子, 检查能否走 (墙? 砖块? 有炸弹?), 能走就更新 pos
- 每帧只能放一颗炸弹, 通过全局 getBombPos() 检查踏上去的位置是否有炸弹



```
// 固定路线往返巡逻，时间驱动
class Monster : public GameObject {
public:
    Monster(GridPosition p, vector<GridPosition> path);
    void update(int dt, const Map& map) override;
    void draw() const override;
private:
    vector<GridPosition> route;
    int routeId = 0;
    int moveTime = 0;
    static const int MOVE_TIME = 450;
};

void Monster::update(int dt, const Map& map) {
    if (!active || route.empty()) return;
    moveTime += dt;
    while (moveTime >= MOVE_TIME) {
        moveTime -= MOVE_TIME;
        int next = (routeId + 1) % route.size();
        if (map.canWalk(route[next])) {
            routeId = next; pos = route[routeId];
        }
    }
}
```

- 每个怪物创建时被赋予一条固定路线，存放在 `vector<GridPosition> route`
- 用 `routeId` 跟踪当前位置，`(routeId + 1) % route.size()` 实现往返循环
- 时间驱动：`moveTime` 累加每帧的 `dt`，达到 **450ms** 才走一步，使移动速度与帧率无关
- 用 `while` 而不是 `if` 累加 `dt`，防止卡顿时一口气走多步
- 如果路线上的砖块还没炸开（`map.canWalk` 返回 `false`），怪物原地等待



```
// 倒计时 2 秒, 由 Game 统一引爆
class Bomb : public GameObject {
public:
    Bomb(GridPosition p);
    void update(int dt, const Map& map) override;
    void draw() const override;
    bool isReady() const { return timer >= FUSE_TIME;
}
    int getSeconds() const;
private:
    int timer = 0;
    static const int FUSE_TIME = 2000;
};
```

- 设置 2 秒引信 (FUSE_TIME = 2000) , timer 累加每帧 dt
- isReady() 返回 timer >= FUSE_TIME, 由 Game 在 update() 中调用, 统一处理引爆时机
- 炸弹自己不引爆, 只负责计时。引爆逻辑在 Game::explodeBombs() 中统一处理
- 游戏同时只能放一颗炸弹 (countBombs() 检查)



```
// 火焰 450ms 后自动消失
class Explosion : public GameObject {
public:
    Explosion(GridPosition center, vector<GridPosition>
blast);
    void update(int dt, const Map& map) override;
    void draw() const override;
    bool hits(GridPosition p) const;
private:
    vector<GridPosition> cells;
    int timer = 0;
    static const int SHOW_TIME = 450;
};

bool Explosion::hits(GridPosition p) const {
    return active && containsPosition(cells, p);
}
```

- 爆炸范围由 Map::getBlast(center, 2) 计算，四方向各传播 2 格
- 三种传播结果：**WALL** → 停止不加入、**BRICK** → 加入并停止（砖块挡住）、**ROAD** → 加入继续
- hits(p) 判断目标是否在爆炸覆盖的格子里，用于碰撞检测
- 火焰显示 **450ms** 后自动将 active 设为 false，由 Game 统一清理



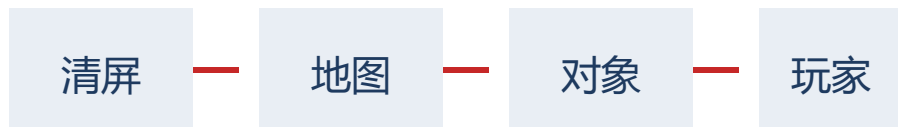
每一帧都重复同一条执行骨架



3 — 双缓冲：解决画面闪烁



直接绘制：用户会看到中间状态



每一步都直接暴露在前台窗口

帧内容尚未完整，画面就被看见

结果：闪烁

双缓冲：完整一帧后再呈现



FlushBatchDraw() 一次翻屏

```
BeginBatchDraw();
```

```
while (running) { update(dt); draw(); FlushBatchDraw(); }
```

```
EndBatchDraw();
```

效果：一帧作为整体更新，用户不再看到绘制过程



同一个问题，经历三步推导

原始方案：每帧移动

```
pos += speed;
```

问题：60 FPS 比 30 FPS
多更新一倍，速度依赖机器

第一次改进：时间门槛

```
if (moveTime >= 450)
```

新问题：卡顿累计 900ms,
if 仍然只补一个逻辑步

最终方案：累计并补步

```
moveTime += dt;  
while (moveTime >= 450) {  
    moveTime -= 450;  
    moveOneStep();  
}
```

双保险

GetTickCount() 计算真实 dt; dt > 100ms 时截断，防止窗口卡顿后瞬移

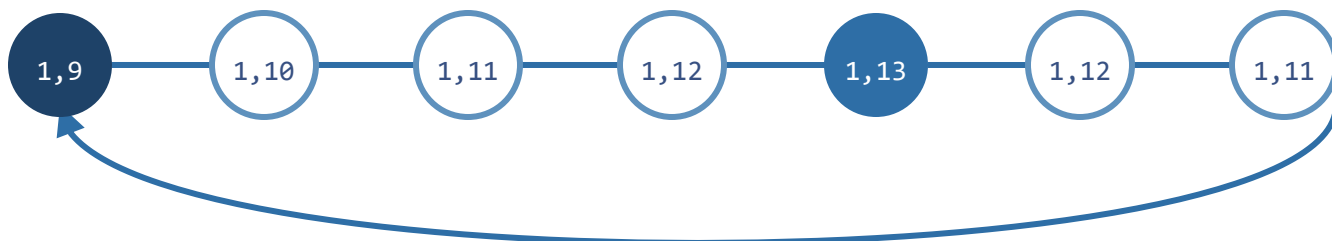
最终效果

渲染帧率可以波动，但怪物按相同的逻辑时间前进

怪物巡路系统——往返式巡逻 + 时间驱动

路线不是“运行时折返”，而是预先展开的圆环

vector<GridPosition> 中已经写入返程坐标，索引只负责循环前进



`next = (routeId + 1) % route.size()` 到尾部后回到索引 0

遇砖块：本次不移动

不改 routeId，也不绕路；
下一帧继续尝试。

6 只怪物：6 条独立路线

五条 8 点、一条 6 点；
不同起点，各走各自的巡逻带。

450ms 时间桶



moveTime 累加真实 dt，满一桶才走一步。
卡顿积累多桶时，**while** 会连续补步。

```
moveTime += dt;
while (moveTime >= MOVE_TIME) {
    moveTime -= MOVE_TIME;
    int next = (routeId + 1) % route.size();
    if (map.canWalk(route[next])) move();
}
```

爆炸传播算法——“BFS 简化版”的准确实现

三种格子，三种语义



停止，不加入
墙本身不受爆炸覆盖



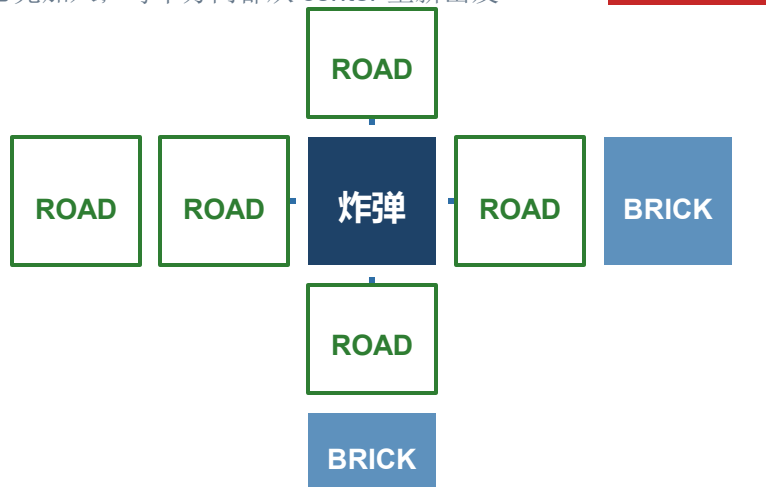
加入，然后停止
砖块被炸，但挡住后方



加入，继续前进
直到 range 用尽

四向各扫描 range = 2

中心先加入；每个方向都从 center 重新出发



为什么严格说不是 BFS?

没有队列，也不会向拐角扩散。
规则天然对应四条独立射线：

```
for direction in {U,D,L,R}
  for step = 1..range
    inspect next tile
```

$O(4 \times \text{range}) = O(\text{range})$

range = 2 时，含中心最多返回 9 格

`vector<GridPosition> cells`

一份 cells，三套系统共同消费。



`breakBrick()`

破坏砖块

`Explosion::draw()`

绘制火焰

`Explosion::hits()`

判断命中

智能指针 + dynamic_cast: 统一管理, 按需区分

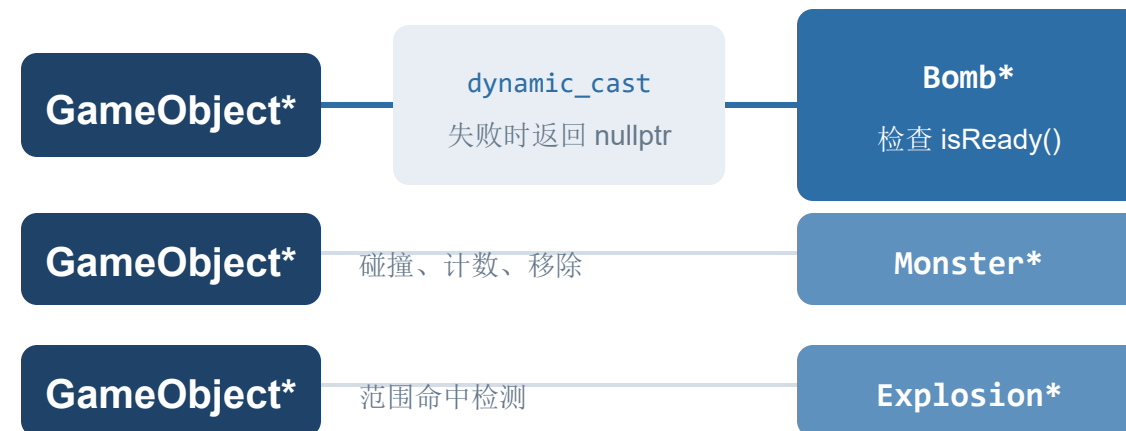
一个容器, 三类动态对象

```
vector<unique_ptr<GameObject>> objects;
```



多态解决**共同生命周期行为**, 不强迫类型特有规则消失。

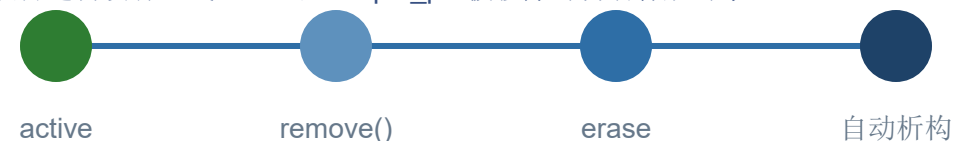
dynamic_cast 是类型特有规则的“筛网”



```
Bomb* bomb = dynamic_cast<Bomb*>(objects[i].get());
if (bomb && bomb->isActive() && bomb->isReady()) {
    // 引爆: 地图计算 → Bomb 失效 → 创建 Explosion
}
```

延迟清理 + RAI

项目逻辑没有显式 delete; unique_ptr 被移除时自动释放对象。



两阶段碰撞检测——顺序决定本帧语义

Game::update(dt) 不是函数清单，而是一条不能随意交换的时间线。



为什么必须有两道碰撞门？



两轮检测让本帧新产生的火焰**本帧就有伤害语义**。

若只在爆炸前检测一次

画面已经出现火焰，但玩家与怪物要到下一帧才受伤。

视觉状态与规则状态错开一帧

感谢老师的评阅

Good Bye!
